

SIMATS ENGINEERING ASSESSMENT TOOLS

COURSE NAME: Theory of Computation

COURSE CODE: CSA1304

COURSE OUTCOME COVERED

CO4: Construct Context-Free Grammars, generate derivations and parse trees to demonstrate the syntactic structure of strings. (BL : 3)

Assessment Tool Weightage: 20%

QUESTIONS: 5 Total Marks:100 Duration : 1 Hour

Industry Problem

Code of Conduct:

I B.JAYANTH(192511375)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

B.JAYANTH

Signature of the student:

SIMATS ENGINEERING ASSESSMENT TOOLS

1.Design and implement a **C-based validation module** that simulates a **Deterministic Finite Automaton (DFA)** to verify input strings in a real-time system. The module should accept only those strings that **begin with the character 'a' and end with the character 'a'**, ensuring compliance with predefined input format rules (e.g., protocol validation, pattern filtering, or secure data transmission).

The system should:

- Process user input dynamically
- Validate strings based on DFA state transitions
- Output whether the given input is **Accepted** or **Rejected**

2. Design a Push Down Automata that accepts the language

$$L = \{w \mid w \in (0 + 1)^* \text{ and } n_0(w) = n_1(w)\}$$

$n_0(w)$ is the number of 0's in w

$n_1(w)$ is the number of 1's in w

3. Design a Pushdown Automata for the language representing a's followed by b's and the number of b's must be twice than that of a's.

$$L = \{a^n b^{2n} \mid n \geq 1\}$$

4. Develop a **C-based computational module** to determine the **ϵ -closure of all states** in a **Non Deterministic Finite Automaton (NFA) with ϵ -transitions**, as part of an automata processing engine used in compiler construction and pattern-matching systems.

5. Develop a **C-based input validation module** that determines whether a given binary string conforms to a specified **Context-Free Grammar (CFG)** used in structured data validation systems.

The grammar is defined as:

- $S \rightarrow 0 A 1$
- $A \rightarrow 0 A \mid 1 A \mid \epsilon$

SIMATS ENGINEERING ASSESSMENT TOOLS

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0– 1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

SIMATS ENGINEERING ASSESSMENT TOOLS

QUESTION 1

DFA-Based String Validation

Given Problem

Design and implement a C-based validation module that simulates a Deterministic Finite Automaton (DFA) to verify input strings in a real-time system.

The DFA must accept only those strings that:

1. Begin with 'a'
2. End with 'a'

The system must:

1. Accept user input dynamically
2. Process the string using DFA state transitions
3. Display Accepted or Rejected

These requirements are directly specified in the assessment document.

1. Language

The required language is:

$L = \{ w \mid w \text{ begins with 'a' and ends with 'a' } \}$

For example:

SIMATS ENGINEERING ASSESSMENT TOOLS

Accepted:

a

aa

aba

abba

aaba

abab a

Rejected:

b

ab

ba

abb

bab

The exact alphabet is not specified in the document, so the program below treats the input as a string and validates the required first and last characters.

2. DFA Definition

The DFA is:

$Q = \{q_0, q_1, q_2, q_{dead}\}$

$\Sigma = \{a, b\}$

SIMATS ENGINEERING ASSESSMENT TOOLS

q_0 = Initial state

$F = \{q_1\}$

Meaning of States

q_0 = Initial state; no input has been processed

q_1 = String starts with 'a' and the current last symbol is 'a'

q_2 = String starts with 'a' but the current last symbol is 'b'

q_{dead} = String does not start with 'a'

3. Transition Table

+-----+-----+-----+			
State	a	b	
+-----+-----+-----+			
q_0	q_1	q_{dead}	
q_1	q_1	q_2	
q_2	q_1	q_2	
q_{dead}	q_{dead}	q_{dead}	
+-----+-----+-----+			

SIMATS ENGINEERING ASSESSMENT TOOLS

Important Transitions

$q_0 \xrightarrow{a} q_1$

$q_0 \xrightarrow{b} q_{dead}$

$q_1 \xrightarrow{a} q_1$

$q_1 \xrightarrow{b} q_2$

$q_2 \xrightarrow{a} q_1$

$q_2 \xrightarrow{b} q_2$

$q_{dead} \xrightarrow{a} q_{dead}$

$q_{dead} \xrightarrow{b} q_{dead}$

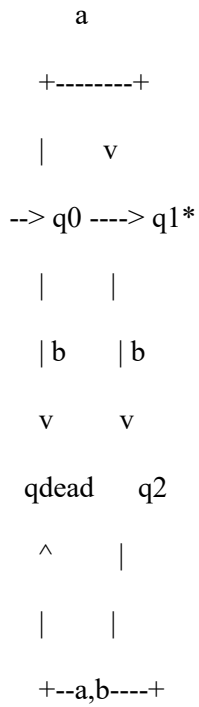
The accepting state is:

q_1

because the string must start with a and its final character must also be a.

4. DFA State Diagram

SIMATS ENGINEERING ASSESSMENT TOOLS



$q1^* \xrightarrow{a} q1$

$q2 \xrightarrow{a} q1$

$q2 \xrightarrow{b} q2$

Here:

$\rightarrow q0$ = Initial state

$q1^*$ = Final/Accepting state

$qdead$ = Dead state

5. Working of the DFA

Consider the input:

SIMATS ENGINEERING ASSESSMENT TOOLS

abba

Process each character:

Initial state = q_0

Input 'a':

$q_0 \rightarrow q_1$

Input 'b':

$q_1 \rightarrow q_2$

Input 'b':

$q_2 \rightarrow q_2$

Input 'a':

$q_2 \rightarrow q_1$

The final state is:

q_1

Since q_1 is the accepting state:

$abba \rightarrow \text{ACCEPTED}$

6. Another Example

Consider:

abb

SIMATS ENGINEERING ASSESSMENT TOOLS

Transitions:

$q_0 \xrightarrow{a} q_1$

$q_1 \xrightarrow{b} q_2$

$q_2 \xrightarrow{b} q_2$

Final state:

q_2

Since q_2 is not an accepting state:

$abb \rightarrow \text{REJECTED}$

7. Algorithm

Step 1: Start the DFA in state q_0 .

Step 2: Read the input string.

Step 3: Read each character one by one.

Step 4: Move to the corresponding state using the transition table.

Step 5: After processing the complete string, check the final state.

Step 6: If the final state is q_1 , print "Accepted".

Step 7: Otherwise, print "Rejected".

SIMATS ENGINEERING ASSESSMENT TOOLS

Step 8: Stop.

8. C Program

```
#include <stdio.h>

#include <string.h>
```

```
int main()
```

```
{
```

```
    char str[100];
```

```
    int state = 0;
```

```
    int i;
```

```
    // Read the input string
```

```
    printf("Enter a string: ");
```

```
    scanf("%s", str);
```

```
    // Process each character using DFA transitions
```

```
    for (i = 0; str[i] != '\0'; i++)
```

```
    {
```

```
        // State q0
```

```
        if (state == 0)
```

```
        {
```

SIMATS ENGINEERING ASSESSMENT TOOLS

```
if (str[i] == 'a')  
    state = 1;  
else  
    state = 3;  
}
```

```
// State q1  
else if (state == 1)  
{  
    if (str[i] == 'a')  
        state = 1;  
    else  
        state = 2;  
}
```

```
// State q2  
else if (state == 2)  
{  
    if (str[i] == 'a')  
        state = 1;  
    else  
        state = 2;  
}
```

SIMATS ENGINEERING ASSESSMENT TOOLS

```
// Dead state qdead

else if (state == 3)

{

    state = 3;

}

}


// q1 is the only accepting state

if (state == 1)

{

    printf("Accepted\n");

}

else

{

    printf("Rejected\n");

}


return 0;

}
```

9. Sample Execution – Accepted

Input

Enter a string: abba

SIMATS ENGINEERING ASSESSMENT TOOLS

State Transitions

$q_0 \rightarrow q_1 \rightarrow q_2 \rightarrow q_2 \rightarrow q_1$

Output

Accepted

10. Sample Execution – Rejected

Input

Enter a string: abb

State Transitions

$q_0 \rightarrow q_1 \rightarrow q_2 \rightarrow q_2$

Final state:

q_2

Since q_2 is not an accepting state:

Rejected

11. Result

The DFA successfully validates the input string by processing each character through the defined state transitions.

Starts with 'a' AND ends with 'a' → Accepted

Otherwise → Rejected

The C program implements the DFA and displays the required **Accepted/Rejected** result as specified in the assessment.

SIMATS ENGINEERING ASSESSMENT TOOLS

QUESTION 2

Design a Pushdown Automaton (PDA) that accepts the language in which the number of 0s in w is equal to the number of 1s in w .

Given Language

$$L = \{ w \in \{0,1\}^* \mid \text{number of 0's in } w = \text{number of 1's in } w \}$$

Answer

The PDA uses the stack to keep track of the unmatched 0s and 1s.

- When a 0 is read and the stack top is 1, pop 1.
- When a 0 is read and there is no unmatched 1, push 0.
- When a 1 is read and the stack top is 0, pop 0.
- When a 1 is read and there is no unmatched 0, push 1.
- If the input is completely processed and only the bottom stack symbol remains, the string is accepted.

1. PDA Definition

$$M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$$

where:

$$Q = \{q_0, q_f\}$$

$$\Sigma = \{0, 1\}$$

SIMATS ENGINEERING ASSESSMENT TOOLS

$\Gamma = \{0, 1, Z0\}$

$q0$ = Initial state

$Z0$ = Initial stack symbol

$F = \{qf\}$

2. Transition Table

Current State | Input | Stack Top | Next State | Stack Operation

 $q0$ | 0 | $Z0$ | $q0$ | Push 0

$q0$ | 0 | 0 | $q0$ | Push 0

$q0$ | 0 | 1 | $q0$ | Pop 1

$q0$ | 1 | $Z0$ | $q0$ | Push 1

$q0$ | 1 | 1 | $q0$ | Push 1

$q0$ | 1 | 0 | $q0$ | Pop 0

$q0$ | ϵ | $Z0$ | qf | Accept

3. State Diagram

SIMATS ENGINEERING ASSESSMENT TOOLS

Copy this structure:

$$0,Z0 \rightarrow 0Z0$$
$$0,0 \rightarrow 00$$
$$1,1 \rightarrow 11$$
$$+ \text{-----} +$$

| |

1

V |

$$((q_0)) \text{-----} +$$
 $\wedge$ |

1

$$+ \text{-----} +$$
$$0,1 \rightarrow \varepsilon$$
$$1,0 \rightarrow \varepsilon$$
$$q_0 \xrightarrow{\varepsilon, Z_0} Z_0 \rightarrow ((qf))$$

Where:

q0 = Initial state

qf = Final state

Z0 = Bottom stack symbol

4. Working of the PDA

Consider the input:

SIMATS ENGINEERING ASSESSMENT TOOLS

0011

Initially:

Input = 0011

Stack = Z0

Step 1: Read 0

There is no unmatched 1, so push 0.

Input = 011

Stack = 0Z0

Step 2: Read 0

Again push 0.

Input = 11

Stack = 00Z0

Step 3: Read 1

The stack top is 0, so pop 0.

Input = 1

Stack = 0Z0

Step 4: Read 1

The stack top is 0, so pop 0.

Input = ϵ

Stack = Z0

The input is completely consumed and there are no unmatched symbols.

Therefore:

0011 $\rightarrow$ ACCEPT

SIMATS ENGINEERING ASSESSMENT TOOLS

5. Instantaneous Description

$(q_0, 0011, Z_0)$

$\vdash (q_0, 011, 0Z_0)$

$\vdash (q_0, 11, 00Z_0)$

$\vdash (q_0, 1, 0Z_0)$

$\vdash (q_0, \varepsilon, Z_0)$

$\vdash (q_f, \varepsilon, Z_0)$

Hence:

$0011 \in L$

6. Rejected Example

Consider:

001

Processing:

$(q_0, 001, Z_0)$

$\vdash (q_0, 01, 0Z_0)$

$\vdash (q_0, 1, 00Z_0)$

$\vdash (q_0, \varepsilon, 0Z_0)$

One 0 remains in the stack.

Therefore:

$001 \rightarrow \text{REJECT}$

because:

SIMATS ENGINEERING ASSESSMENT TOOLS

Number of 0's = 2

Number of 1's = 1

7. Result

The designed PDA accepts a string when:

Number of 0's = Number of 1's

and rejects it when the two counts are different.

0011 → ACCEPT

0101 → ACCEPT

1100 → ACCEPT

001 → REJECT

0001 → REJECT

QUESTION 3

Design a Pushdown Automaton for the language:

$$L = \{ a^n b^{2n} \mid n \geq 1 \}$$

The number of bs must be **twice the number of as**. This is the language specified in Question 3 of the assessment document.

1. Idea of the PDA

SIMATS ENGINEERING ASSESSMENT TOOLS

For every a read from the input, the PDA must account for **two bs**.

Therefore:

1 a $\rightarrow$ 2 b's

2 a's $\rightarrow$ 4 b's

3 a's $\rightarrow$ 6 b's

The PDA uses the stack to store **two markers for every a**.

When an a is read:

Push two X symbols onto the stack.

When a b is read:

Pop one X from the stack.

The string is accepted when all the X symbols are removed after the complete input is processed.

2. PDA Definition

$M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$

where:

$Q = \{q_0, q_1, q_f\}$

$\Sigma = \{a, b\}$

$\Gamma = \{X, Z_0\}$

q_0 = Initial state

SIMATS ENGINEERING ASSESSMENT TOOLS

Z_0 = Initial stack symbol

$F = \{q_f\}$

State Meaning

q_0 = Reading a's and pushing two X's for every a

q_1 = Reading b's and popping one X for every b

q_f = Final accepting state

3. Transition Table

Current State	Input	Stack Top	Next State	Stack Operation
q_0	a	Z_0	q_0	Push XXZ_0
q_0	a	X	q_0	Push XXX
q_0	b	X	q_1	Pop X
q_1	b	X	q_1	Pop X
q_1	ϵ	Z_0	q_f	Accept

Important Point

SIMATS ENGINEERING ASSESSMENT TOOLS

The transition:

$q_0, a, Z_0 \rightarrow q_0, XXZ_0$

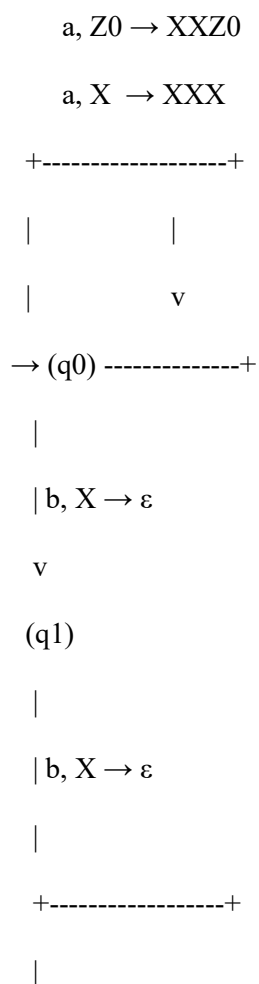
means the first a causes **two X symbols** to be placed on the stack.

For subsequent as:

$q_0, a, X \rightarrow q_0, XXX$

adds two more Xs while preserving the existing stack symbol.

4. State Diagram



SIMATS ENGINEERING ASSESSMENT TOOLS

| ϵ , $Z_0 \rightarrow Z_0$

v

((qf))

q0 = Initial state

q1 = Reading b's

qf = Final state

5. Working of the PDA

Consider:

w = aabbbb

Here:

Number of a's = 2

Number of b's = 4

Since:

$$4 = 2 \times 2$$

the string should be accepted.

Step 1: Read first a

Push two Xs.

Input = abbbb

Stack = XXZ0

Step 2: Read second a

Push two more Xs.

Input = bbbb

SIMATS ENGINEERING ASSESSMENT TOOLS

Stack = XXXXZ0

Now there are four Xs in the stack.

Step 3: Read first b

Pop one X.

Input = bbb

Stack = XXXZ0

Step 4: Read second b

Pop one X.

Input = bb

Stack = XXZ0

Step 5: Read third b

Pop one X.

Input = b

Stack = XZ0

Step 6: Read fourth b

Pop one X.

Input = ϵ

Stack = Z0

All input has been consumed and all X markers have been removed.

Therefore:

aabbbb $\rightarrow$ ACCEPT

6. Instantaneous Description

SIMATS ENGINEERING ASSESSMENT TOOLS

For the string aabbbb:

$(q_0, aabbbb, Z_0)$

$\vdash (q_0, abbbb, XXZ_0)$

$\vdash (q_0, bbbb, XXXXZ_0)$

$\vdash (q_1, bbb, XXXZ_0)$

$\vdash (q_1, bb, XXZ_0)$

$\vdash (q_1, b, XZ_0)$

$\vdash (q_1, \epsilon, Z_0)$

$\vdash (q_f, \epsilon, Z_0)$

Therefore:

$aabbbb \in L$

7. Rejected Example

Consider:

$w = aabbb$

Here:

Number of a's = 2

Number of b's = 3

But the required number of bs is:

$$2 \times 2 = 4$$

After processing the three bs:

Stack = XZ₀

One X remains.

SIMATS ENGINEERING ASSESSMENT TOOLS

Therefore:

aabbb $\rightarrow$ REJECT

8. Examples

Input String	Number of a's	Required b's	Actual b's	Result
ab b	1	2	2	Accepted
aabbbb	2	4	4	Accepted
aa bbbbb	3	6	6	Accepted
aabbb	2	4	3	Rejected
aaabbbb	3	6	5	Rejected

For clarity, the accepted strings are:

ab

aabbbb

aaabbbbb

aaaabbbbbbb

9. Result

The designed PDA accepts strings in which all as occur before all bs and the number of bs is exactly twice the number

SIMATS ENGINEERING ASSESSMENT TOOLS

of as.

$$L = \{ a^n b^{2n} \mid n \geq 1 \}$$

Thus:

$$\text{Number of b's} = 2 \times \text{Number of a's}$$

The PDA accepts aabbbb and rejects strings such as aabbb.

QUESTION 4

Develop a C-based computational module to determine the **ϵ -closure of all states** in a Non-Deterministic Finite Automaton (NFA) with ϵ -transitions, as part of an automata processing engine used in compiler construction and pattern-matching systems.

1. Definition of ϵ -Closure

The **ϵ -closure of a state** is the set of all states that can be reached from that state using **zero or more ϵ -transitions**.

The state itself is always included in its ϵ -closure.

For example:

$$q_0 \xrightarrow{\epsilon} q_1 \xrightarrow{\epsilon} q_2$$

Then:

$$\epsilon\text{-closure}(q_0) = \{q_0, q_1, q_2\}$$

$$\epsilon\text{-closure}(q_1) = \{q_1, q_2\}$$

$$\epsilon\text{-closure}(q_2) = \{q_2\}$$

SIMATS ENGINEERING ASSESSMENT TOOLS

An ϵ -transition changes the state without consuming an input symbol.

2. Method Used

The program represents the ϵ -transitions using an **adjacency matrix**.

If there is an ϵ -transition from state i to state j :

$$\text{epsilon}[i][j] = 1$$

Otherwise:

$$\text{epsilon}[i][j] = 0$$

For each state:

1. Start with the state itself.
2. Follow every possible ϵ -transition.
3. Continue following ϵ -transitions from newly reached states.
4. Store every reachable state.
5. Stop when no new state can be reached.

3. Algorithm

Step 1: Read the number of states.

Step 2: Read the ϵ -transition matrix.

Step 3: For each state i :

- a. Initially include i in its ϵ -closure.

SIMATS ENGINEERING ASSESSMENT TOOLS

- b. Place i in a stack.
- c. Remove one state from the stack.
- d. Check all ϵ -transitions from that state.
- e. If a new state is reached, add it to the closure
and push it into the stack.
- f. Repeat until the stack becomes empty.

Step 4: Display the ϵ -closure of every state.

Step 5: Stop.

4. C Program

```
#include <stdio.h>
```

```
#define MAX 20
```

```
int epsilon[MAX][MAX];
```

```
int closure[MAX][MAX];
```

```
int n;
```

```
void findEpsilonClosure(int state)
```

```
{
```

```
    int stack[MAX];
```

SIMATS ENGINEERING ASSESSMENT TOOLS

```
int top = -1;
```

```
int current;
```

```
int i;
```

```
closure[state][state] = 1;
```

```
stack[++top] = state;
```

```
while (top != -1)
```

```
{
```

```
    current = stack[top--];
```

```
    for (i = 0; i < n; i++)
```

```
    {
```

```
        if (epsilon[current][i] == 1 &&
```

```
            closure[state][i] == 0)
```

```
        {
```

```
            closure[state][i] = 1;
```

```
            stack[++top] = i;
```

```
        }
```

```
    }
```

```
}
```

```
}
```

```
int main()
```

SIMATS ENGINEERING ASSESSMENT TOOLS

```
{  
  
    int i, j;  
  
  
  
    printf("Enter number of states: ");  
  
    scanf("%d", &n);  
  
  
  
    printf("\nEnter epsilon transition matrix:\n");  
  
  
  
    for (i = 0; i < n; i++)  
    {  
        for (j = 0; j < n; j++)  
        {  
            scanf("%d", &epsilon[i][j]);  
        }  
    }  
  
  
  
    for (i = 0; i < n; i++)  
    {  
        findEpsilonClosure(i);  
    }  
  
  
  
    printf("\nEpsilon-Closure of all states:\n");
```


SIMATS ENGINEERING ASSESSMENT TOOLS

```
for (i = 0; i < n; i++)  
{  
    printf("E-Closure(q%d) = { ", i);  
  
    for (j = 0; j < n; j++)  
    {  
        if (closure[i][j] == 1)  
        {  
            printf("q%d ", j);  
        }  
    }  
  
    printf("}\n");  
}  
  
return 0;  
}
```

5. Example NFA

Consider the following ε -transitions:

$q_0 \xrightarrow{\varepsilon} q_1$

$q_1 \xrightarrow{\varepsilon} q_2$

SIMATS ENGINEERING ASSESSMENT TOOLS

$q_2 \xrightarrow{\epsilon} q_3$

There are no other ϵ -transitions.

Transition Table

From State	ϵ -Transition To
q_0	q_1
q_1	q_2
q_2	q_3
q_3	None

Therefore:

$$E\text{-Closure}(q_0) = \{q_0, q_1, q_2, q_3\}$$

$$E\text{-Closure}(q_1) = \{q_1, q_2, q_3\}$$

$$E\text{-Closure}(q_2) = \{q_2, q_3\}$$

$$E\text{-Closure}(q_3) = \{q_3\}$$

6. Matrix Representation

For the above NFA, the ϵ -transition matrix is:

SIMATS ENGINEERING ASSESSMENT TOOLS

State	q0	q1	q2	q3
q0	0	1	0	0
q1	0	0	1	0
q2	0	0	0	1
q3	0	0	0	0

Here:

1 = ϵ -transition exists

0 = ϵ -transition does not exist

7. Sample Input

Enter number of states: 4

Enter epsilon transition matrix:

0 1 0 0

0 0 1 0

0 0 0 1

0 0 0 0

8. Sample Output

Epsilon-Closure of all states:

E-Closure(q0) = { q0 q1 q2 q3 }

SIMATS ENGINEERING ASSESSMENT TOOLS

$$E\text{-Closure}(q_1) = \{ q_1 \ q_2 \ q_3 \}$$

$$E\text{-Closure}(q_2) = \{ q_2 \ q_3 \}$$

$$E\text{-Closure}(q_3) = \{ q_3 \}$$

9. Working of ϵ -Closure

For q_0 :

q_0

$\rightarrow_{\epsilon} q_1$

$\rightarrow_{\epsilon} q_2$

$\rightarrow_{\epsilon} q_3$

Therefore:

$$E\text{-Closure}(q_0) = \{q_0, q_1, q_2, q_3\}$$

For q_1 :

q_1

$\rightarrow_{\epsilon} q_2$

$\rightarrow_{\epsilon} q_3$

Therefore:

$$E\text{-Closure}(q_1) = \{q_1, q_2, q_3\}$$

For q_2 :

q_2

$\rightarrow_{\epsilon} q_3$

Therefore:

$$E\text{-Closure}(q_2) = \{q_2, q_3\}$$

SIMATS ENGINEERING ASSESSMENT TOOLS

For q_3 :

q_3

There is no outgoing ϵ -transition.

Therefore:

$$E\text{-Closure}(q_3) = \{q_3\}$$

10. Result

The C-based module successfully computes the ϵ -closure of **every state** of an NFA by following all possible ϵ -transitions without consuming an input symbol.

$$E\text{-Closure}(q_0) = \{q_0, q_1, q_2, q_3\}$$

$$E\text{-Closure}(q_1) = \{q_1, q_2, q_3\}$$

$$E\text{-Closure}(q_2) = \{q_2, q_3\}$$

$$E\text{-Closure}(q_3) = \{q_3\}$$

QUESTION 5

Develop a C-based input validation module that determines whether a given binary string conforms to the specified Context-Free Grammar (CFG) used in structured data validation systems.

The grammar is:

$$S \rightarrow 0A1$$

$$A \rightarrow 0A \mid 1A \mid \epsilon$$

1. Understanding the Grammar

SIMATS ENGINEERING ASSESSMENT TOOLS

Given:

$$S \rightarrow 0A1$$

$$A \rightarrow 0A \mid 1A \mid \varepsilon$$

The first production:

$$S \rightarrow 0A1$$

means every generated string must:

Start with 0

End with 1

The production:

$$A \rightarrow 0A \mid 1A$$

allows any number of 0s and 1s in the middle.

Finally:

$$A \rightarrow \varepsilon$$

terminates the derivation.

Therefore, the language generated by this CFG is:

$$L = \{ 0w1 \mid w \in \{0,1\}^* \}$$

In simple words:

The string must begin with 0 and end with 1.

2. Examples

Input String	Starts with 0	Ends with 1	Result
01	Yes	Yes	Accepted

SIMATS ENGINEERING ASSESSMENT TOOLS

Input String	Starts with 0	Ends with 1	Result
001	Yes	Yes	Accepted
011	Yes	Yes	Accepted
0101	Yes	Yes	Accepted
01101	Yes	Yes	Accepted
10	No	No	Rejected
00	Yes	No	Rejected
11	No	Yes	Rejected
101	No	Yes	Rejected

3. Derivation

Consider the string:

0101

Start with:

S

Apply:

$S \rightarrow 0A1$

Therefore:

S

$\rightarrow 0A1$

To generate the middle 01, use:

$A \rightarrow 0A$

SIMATS ENGINEERING ASSESSMENT TOOLS

Then:

0A1

$\rightarrow 00A1$

Now use:

$A \rightarrow 1A$

Therefore:

00A1

$\rightarrow 001A1$

Finally:

$A \rightarrow \varepsilon$

Therefore:

001A1

$\rightarrow 001\varepsilon1$

$\rightarrow 0011$

For a simpler accepted string such as 0101, the correct derivation is:

S

$\rightarrow 0A1$

$\rightarrow 01A1$

$\rightarrow 01\varepsilon1$

$\rightarrow 011$

This shows that the grammar generates strings beginning with 0 and ending with 1.

4. Validation Logic

SIMATS ENGINEERING ASSESSMENT TOOLS

The C program needs to check three conditions:

1. The input must contain only 0 and 1.
2. The first character must be 0.
3. The last character must be 1.

If all three conditions are satisfied:

Accepted

Otherwise:

Rejected

5. Algorithm

Step 1: Read the binary string.

Step 2: Check whether the string is empty.

Step 3: Check every character.

If a character is not 0 or 1, reject the string.

Step 4: Check whether the first character is 0.

Step 5: Check whether the last character is 1.

Step 6: If all conditions are satisfied, print Accepted.

SIMATS ENGINEERING ASSESSMENT TOOLS

Step 7: Otherwise, print Rejected.

Step 8: Stop.

6. C Program

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main()
```

```
{
```

```
    char str[100];
```

```
    int i;
```

```
    int valid = 1;
```

```
    printf("Enter a binary string: ");
```

```
    scanf("%s", str);
```

```
    for (i = 0; str[i] != '\0'; i++)
```

```
    {
```

```
        if (str[i] != '0' && str[i] != '1')
```

```
        {
```

```
            valid = 0;
```

SIMATS ENGINEERING ASSESSMENT TOOLS

```
        break;

    }

}

if (valid && str[0] != '0')

{

    valid = 0;

}

if (valid && str[strlen(str) - 1] != '1')

{

    valid = 0;

}

if (valid)

{

    printf("Accepted\n");

}

else

{

    printf("Rejected\n");

}

return 0;
```

SIMATS ENGINEERING ASSESSMENT TOOLS

}

7. Sample Execution – Accepted

Input

Enter a binary string: 0101

Checking

First character = 0

Last character = 1

All characters = 0 or 1

Therefore:

Accepted

8. Sample Execution – Rejected

Input

Enter a binary string: 1010

Checking

First character = 1

But the grammar requires:

First character = 0

Therefore:

Rejected

SIMATS ENGINEERING ASSESSMENT TOOLS

9. Another Derivation

For the string:

001

The derivation is:

S

$\rightarrow 0A1$

$\rightarrow 00A1$

$\rightarrow 00\varepsilon 1$

$\rightarrow 001$

Therefore:

$001 \in L$

For:

10

there is no valid derivation because the string does not begin with 0 and end with 1.

Therefore:

$10 \notin L$

10. Result

The C-based validation module successfully checks whether a binary string conforms to the given CFG:

$S \rightarrow 0A1$

$A \rightarrow 0A \mid 1A \mid \varepsilon$

The generated language is:

SIMATS ENGINEERING ASSESSMENT TOOLS

$$L = \{0w1 \mid w \in \{0,1\}^*\}$$

Hence, the program accepts binary strings that **begin with 0 and end with 1**, while rejecting strings that do not satisfy the grammar.